AgentFlow 完整复现指南（定稿）

---

0. 读前须知

关于模型名称

作业要求的 Qwen3.5-0.8B / 2B / 4B / 9B / 27B 均已于 2026 年 2-3 月公开发布，HuggingFace 可直接下载。本文直接使用对应型号，无需近似映射：

作业写法	HuggingFace 路径
0.8B	Qwen/Qwen3.5-0.8B
2B	Qwen/Qwen3.5-2B
4B	Qwen/Qwen3.5-4B
9B	Qwen/Qwen3.5-9B
27B	Qwen/Qwen3.5-27B

关于复现目标

本文 Step 2 复现的是论文表格中 AgentFlow（无 Flow-GRPO）行，即倒数第二行，目标 Avg=47.2 / GAIA=17.2，不是 AgentFlow (w/ Flow-GRPO) 行。

关于训练框架

安装最新版 PEFT / TRL / Transformers 满足作业"使用最新库版本"的要求。Flow-GRPO 的多轮 agent 训练采用 AgentFlow repo 自身流程完成（基于 verl），TRL 面向单步 sequence-level RL 设计，不支持 multi-turn rollout，verl 才能实现 trajectory-level 优化。Step 5 以 TinyZero 复现为起点（见 5.1），在此基础上适配 AgentFlow 的 LoRA 训练流程。

关于 API Keys

本系统需要以下 keys，缺少任何一个会导致对应功能失效：

Key	用途	缺失后果
OPENAI_API_KEY	LLM-as-judge 评判答案	答案正确率无法计算
GOOGLE_API_KEY	Google Search 工具	搜索类 benchmark 结果很差
TOGETHER_API_KEY 或 DASHSCOPE_API_KEY	Executor/Verifier/Generator	agent loop 无法运行

如无 OPENAI_API_KEY：可用 DashScope 的 Qwen2.5-7B 替代 judge，在 `.env` 里设 `JUDGE_ENGINE=dashscope`，评分逻辑相同。

关于 vLLM 与 Qwen3.5 兼容性

标准 pip 版 vLLM 不支持 Qwen3.5 架构（Gated Delta Networks）。必须从主分支安装：

```bash
uv pip install vllm --torch-backend=auto \
  --extra-index-url https://download.pytorch.org/whl/cu124
```

SGLang 同样支持，作为备选方案（见 2.2）。

关于模块固定策略

固定 Executor / Verifier / Generator 使用 Qwen2.5-7B-Instruct（通过 Together/DashScope API），只训练和替换 Planner 模块。论文中这三个模块同样使用 Qwen2.5-7B-Instruct，与此一致。

成本说明：每道题平均 5 轮 agent loop，每轮调用 3 个模块，5 个 benchmark 合计约数千次 API call，按 TogetherAPI 定价估计约 $20–50。

关于新 Benchmark 选择

Step 4 和 Step 6 选择 **Text-to-SQL（Spider 数据集）**，理由：

- 与论文原始任务（QA、搜索）类型显著不同，满足"significantly different"要求
- 评测完全确定性（execution accuracy），无需 LLM-as-judge
- Spider 是标准公开数据集，HuggingFace 可直接拉取
- 工具扩展最小：只需新增一个 `execute_sql` 工具

关于架构

```
Query
|
▼
Planner ← 唯一被训练的模块。接受 (query, memory, toolset), 输出 JSON action
|         {thought: ..., tool: <工具名>, input: <工具输入>}
▼
Executor ← 固定, 调用实际工具 (搜索、代码执行等)
|
▼
Verifier ← 固定, 判断执行结果是否满足子目标 (返回 0/1)
|           若 0 → 更新 memory → 回到 Planner (下一轮)
|           若 1 → 结束循环
▼
Generator ← 固定, 根据最终 memory 生成人类可读的答案
```

Step 1：安装环境并跑通 repo

1.1 安装

```bash
git clone https://github.com/lupantech/AgentFlow
cd AgentFlow
# repo 要求 Python 3.11
bash setup.sh
source .venv/bin/activate

# 升级到最新库版本 (满足作业要求)
pip install --upgrade transformers peft trl accelerate

# vLLM 主分支版本 (支持 Qwen3.5 架构)
uv pip install vllm --torch-backend=auto \
  --extra-index-url https://download.pytorch.org/whl/cu124

# 额外安装: Modal (GPU 推理)、verl (训练框架)
pip install modal
pip install verl
```

1.2 配置 API Keys

```bash
cp agentflow/agentflow/.env.template agentflow/agentflow/.env
# 编辑 agentflow/agentflow/.env，填入：
OPENAI_API_KEY=sk-...           # LLM-as-judge（可用 DashScope 替代）
GOOGLE_API_KEY=...              # Google Search 工具
TOGETHER_API_KEY=...            # Executor/Verifier/Generator
# 国内用户改用 DASHSCOPE_API_KEY
MODAL_TOKEN_ID=...
MODAL_TOKEN_SECRET=...
```

Modal 账号初始化：`modal token new`

1.3 验证环境能跑通

```bash
python quick_start.py
# 期望看到 ==> ✅ Query Solved!

cd agentflow/agentflow && bash ./tools/test_all_tools.sh
# 期望：✅ All tests passed
```

Step 2：复现 AgentFlow 无 Flow-GRPO 结果（Qwen2.5-7B-Instruct）

目标：Bamboogle 58.4 / 2Wiki 60.0 / HotpotQA 51.3 / Musique 19.2 / GAIA 17.2

2.1 先用 repo 原生方式跑通 benchmark

```bash
cd test/bamboogle
cat run.sh  # 确认 llm_engine_name 已设为 "dashscope" 或 "together"
bash run.sh
# 结果在 results/finalscore_*.log

cd ../2wiki && bash run.sh
cd ../hotpotqa && bash run.sh
cd ../musique && bash run.sh
cd ../gaia && bash run.sh
```

2.2 切换到 Modal vLLM 部署

新建 `modal_serve.py`：

```python
```

```python
# modal_serve.py
import modal

image = (
    modal.Image.from_registry(
        "nvidia/cuda:12.4.0-devel-ubuntu22.04", add_python="3.11"
    )
    # 必须从主分支安装以支持 Qwen3.5 Gated Delta Networks 架构
    .run_commands(
        "pip install uv",
        "uv pip install vllm --torch-backend=auto "
        "--extra-index-url https://download.pytorch.org/whl/cu124",
    )
    .pip_install("huggingface_hub", "transformers")
)

app = modal.App("agentflow-planner")

@app.cls(
    gpu=modal.gpu.A100(memory=80),
    image=image,
    timeout=7200,
    keep_warm=1,
    secrets=[modal.Secret.from_name("huggingface-secret")],
)
class PlannerServer:
    model_id: str = modal.parameter(default="Qwen/Qwen2.5-7B-Instruct")
    is_qwen35: bool = modal.parameter(default=False)

    @modal.enter()
    def load_model(self):
        from vllm import LLM, SamplingParams
        from transformers import AutoTokenizer
        self.tok = AutoTokenizer.from_pretrained(
            self.model_id, trust_remote_code=True
        )
        self.llm = LLM(
            model=self.model_id,
            dtype="bfloat16",
            max_model_len=32768,
            trust_remote_code=True,
            gpu_memory_utilization=0.90,
        )
```

```
        extra = {}
        if self.is_qwen35:
            # Qwen3.5 thinking mode 会破坏 Planner 的 JSON 解析，必须关闭
            extra = {"chat_template_kwargs": {"enable_thinking": False}}
        self.sp = SamplingParams(
            temperature=0.0, max_tokens=2048, extra_body=extra
        )

    @modal.web_endpoint(method="POST", label="chat")
    def chat(self, request: dict) -> dict:
        prompt = self.tok.apply_chat_template(
            request["messages"],
            tokenize=False,
            add_generation_prompt=True,
        )
        out = self.llm.generate([prompt], self.sp)
        text = out[0].outputs[0].text
        return {"choices": [{"message": {"role": "assistant", "content": text}}]}
```

部署：

```bash
modal deploy modal_serve.py
# 记录 endpoint URL：
# https://<workspace>--agentflow-planner-plannerserver-chat.modal.run
```

2.3 把 Modal endpoint 接入 AgentFlow

新建 `agentflow/agentflow/engine/modal_engine.py`：

```python
```

```python
import os, requests
from .base import BaseLLMEngine

class ModalEngine(BaseLLMEngine):
    def __init__(self, model_id: str = "", temperature: float = 0.0, **kwargs):
        self.url = os.environ["MODAL_PLANNER_URL"]
        self.temperature = temperature

    def chat(self, messages: list[dict]) -> str:
        resp = requests.post(
            self.url,
            json={"messages": messages, "temperature": self.temperature},
            timeout=180,
        )
        resp.raise_for_status()
        return resp.json()["choices"][0]["message"]["content"]
```

在 `agentflow/agentflow/engine/factory.py` 加分支：

```python
elif model_string.startswith("modal"):
    from .modal_engine import ModalEngine
    return ModalEngine(model_id=model_string, temperature=temperature)
```

```bash
export MODAL_PLANNER_URL="https://<workspace>--agentflow-planner-plannerserver-chat.
cd test
bash bamboogle/run.sh && bash 2wiki/run.sh && bash hotpotqa/run.sh \
  && bash musique/run.sh && bash gaia/run.sh
```

Step 3：Qwen3.5 各尺寸模型跑同样 5 个 Benchmark（无训练）

每个模型重新部署 Modal 实例，只换 `model_id` 参数和 `is_qwen35=true`。

显存参考：

模型	bfloat16 显存	单 A100-80G
Qwen3.5-0.8B	~2GB	✅
Qwen3.5-2B	~5GB	✅
Qwen3.5-4B	~9GB	✅
Qwen3.5-9B	~19GB	✅
Qwen3.5-27B	~55GB	✅

```bash
# Qwen3.5-0.8B 示例:
# modal deploy modal_serve.py → model_id=Qwen/Qwen3.5-0.8B, is_qwen35=true
export MODAL_PLANNER_URL="https://...qwen35-0.8b...modal.run"
cd test
bash bamboogle/run.sh && bash 2wiki/run.sh && bash hotpotqa/run.sh \
  && bash musique/run.sh && bash gaia/run.sh

# 对 2B / 4B / 9B / 27B 重复以上操作
```

Step 4：新 Benchmark —— Spider（Text-to-SQL）

4.1 准备数据

```bash
pip install datasets sqlparse

# Spider 数据库文件（SQLite）需要手动下载
# 去 https://yale-nlp.github.io/spider/ 下载 spider.zip，解压到 data/spider/
# 目录结构：data/spider/database/<db_id>/<db_id>.sqlite
```

4.2 注册 execute_sql 工具

AgentFlow 通过 tool registry 调工具：Planner 输出 `{"tool": "execute_sql", "input": "..."}` → Executor 按名字在 registry 查找并调用。不注册直接报错。

第一步：新建 `agentflow/agentflow/tools/execute_sql_tool.py`：

```python
import sqlite3, os

def execute_sql(sql_query: str) -> str:
    db_path = os.environ.get("SPIDER_DB_PATH", "")
    if not db_path:
        return "Error: SPIDER_DB_PATH environment variable not set."
    try:
        conn = sqlite3.connect(db_path)
        cur = conn.cursor()
        cur.execute(sql_query)
        rows = cur.fetchmany(20)
        cols = [d[0] for d in (cur.description or [])]
        conn.close()
        if not rows:
            return "Query returned no results."
        header = " | ".join(cols)
        lines = [header, "-" * len(header)]
        lines += [" | ".join(str(v) for v in row) for row in rows]
        return "\n".join(lines)
    except Exception as e:
        return f"SQL Error: {e}"

TOOL_META = {
    "name": "execute_sql",
    "description": (
        "Execute a SQL query against the current SQLite database and return results.
        "Use this tool to explore data and verify your SQL logic."
    ),
    "input_description": "A valid SQL query string.",
    "function": execute_sql,
}
```

第二步：在 `agentflow/agentflow/tools/__init__.py` 注册：

```python
```

```python
from .execute_sql_tool import TOOL_META as execute_sql_meta

AVAILABLE_TOOLS = {
    # ... 原有工具 ...
    "execute_sql": execute_sql_meta,
}
```

第三步：初始化 solver 时加入工具：

```python
solver = construct_solver(
    llm_engine_name=llm_engine_name,
    tools=["google_search", "wikipedia_search", "execute_sql"],
)
```

4.3 实现评测脚本

新建 `test/text2sql/agentflow_spider.py`：

```python
```

```python
import sqlite3, re
from datasets import load_dataset
from agentflow.agentflow.solver import construct_solver

dataset = load_dataset("yale-nlp/spider", split="validation")

def get_schema(db_path: str) -> str:
    conn = sqlite3.connect(db_path)
    cur = conn.cursor()
    cur.execute("SELECT name FROM sqlite_master WHERE type='table'")
    tables = [r[0] for r in cur.fetchall()]
    lines = []
    for t in tables:
        cur.execute(f"PRAGMA table_info({t})")
        cols = [f"{r[1]} ({r[2]})" for r in cur.fetchall()]
        lines.append(f"Table {t}: {', '.join(cols)}")
    conn.close()
    return "\n".join(lines)

def execute_sql(sql: str, db_path: str) -> list:
    try:
        conn = sqlite3.connect(db_path)
        cur = conn.cursor()
        cur.execute(sql)
        rows = cur.fetchall()
        conn.close()
        return rows
    except Exception as e:
        return [("ERROR", str(e))]

def extract_sql(text: str) -> str:
    m = re.search(r"```sql\n(.*?)\n```", text, re.DOTALL)
    if m:
        return m.group(1).strip()
    lines = [l for l in text.split("\n") if l.strip().upper().startswith("SELECT")]
    return lines[-1].strip() if lines else text.strip()

def run_spider_eval(llm_engine_name: str = "modal",
                    db_dir: str = "data/spider/database"):
    solver = construct_solver(llm_engine_name=llm_engine_name)
    correct, total = 0, 0
    for sample in dataset:
        question = sample["question"]
```

```
        gold_sql = sample["query"]
        db_id = sample["db_id"]
        db_path = f"{db_dir}/{db_id}/{db_id}.sqlite"
        schema = get_schema(db_path)
        prompt = (
            f"Database schema:\n{schema}\n\n"
            f"Question: {question}\n\n"
            "Use execute_sql tool to explore the database and find the answer. "
            "Output the final SQL query in a ```sql``` block."
        )
        output = solver.solve(prompt)
        pred_sql = extract_sql(output["direct_output"])
        pred_rows = execute_sql(pred_sql, db_path)
        gold_rows = execute_sql(gold_sql, db_path)
        if set(map(tuple, pred_rows)) == set(map(tuple, gold_rows)):
            correct += 1
        total += 1
        if total % 50 == 0:
            print(f"[{total}/{len(dataset)}] Exec Acc: {correct/total:.3f}")
    print(f"\nFinal Execution Accuracy: {correct/total:.3f} ({correct}/{total})")

if __name__ == "__main__":
    run_spider_eval()
```

4.4 运行

```bash
# 对每个 Qwen3.5 尺寸分别运行（换 MODAL_PLANNER_URL 即可）
python test/text2sql/agentflow_spider.py
```

4.5 AgentFlow Loop 示例 Trace（提交时附上）

为证明 Spider 评测确实运行在完整 AgentFlow 系统内（而非单纯 prompt 模型吐 SQL），报告中需附一条完整 trace：

```
Question: "How many singers are there?"
Database: concert_singer

[Turn 1] Planner →
  {
    "thought": "I need to count distinct singers in the database.",
    "tool": "execute_sql",
    "input": "SELECT COUNT(*) FROM singer"
  }

[Turn 1] Executor →
  COUNT(*)
  --------
  5

[Turn 1] Verifier →
  Result matches sub-goal (count retrieved). reward=1 → exit loop.

[Generator] →
  Final SQL: SELECT COUNT(*) FROM singer
  Answer: There are 5 singers.
```

Step 5：Flow-GRPO + LoRA 训练 Qwen3.5-0.8B

> **Note**：Step 5 以 TinyZero 复现（Part 1 作业）为起点，在其 LoRA + GRPO 训练框架基础上，适配 AgentFlow 的 multi-turn rollout 流程和训练数据。训练数据选用 AgentFlow repo 原始流程（NQ + DeepMath-103K），与论文训练设置保持一致，评测目标为论文要求的 5 个 benchmark。

Flow-GRPO 本质：GRPO 对每个 query 并行采样 G=8 条完整轨迹，组内相对排名归一化成 advantage，广播到轨迹每一步 Planner action，用 PPO clip 目标更新 LoRA 权重。

5.1 从 TinyZero 迁移训练配置

TinyZero 已经完成了 LoRA + GRPO 的基础验证（countdown task）。这里复用其训练结构，替换模型和数据：

```bash
```

```
# 下载训练数据（AgentFlow repo 原始流程）
python data/get_train_data.py   # NQ + DeepMath-103K → combined_train.parquet
python data/aime24_data.py      # 验证集 aime24.parquet

# 复制 TinyZero 的 LoRA 配置为起点
cp <tinyzero_repo>/train/config_lora.yaml train/config_qwen35_lora.yaml
```

5.2 修改训练配置

编辑 `train/config_qwen35_lora.yaml`：

```yaml
```

```yaml
actor_rollout_ref:
  model:
    path: Qwen/Qwen3.5-0.8B
    use_remove_padding: true
    enable_gradient_checkpointing: true
    # LoRA 参数（必须在 actor_rollout_ref.model 下，不是顶层）
    lora_rank: 8
    lora_alpha: 32
    lora_dropout: 0.05
    lora_target_modules:
      - q_proj
      - k_proj
      - v_proj
      - o_proj
      - gate_proj
      - up_proj
      - down_proj

  rollout:
    name: vllm
    n: 8  # G=8，每个 query 采样 8 条轨迹
    gpu_memory_utilization: 0.6
    chat_template_kwargs:
      enable_thinking: false  # Qwen3.5 必须关闭 thinking mode

  actor:
    optim:
      lr: 1.0e-5
    ppo_mini_batch_size: 16
    clip_ratio_low: 0.2
    clip_ratio_high: 0.3
    use_kl_loss: true
    kl_loss_coef: 0.001

data:
  train_files: data/train/combined_train.parquet
  val_files: data/val/aime24.parquet
  train_batch_size: 64
  max_prompt_length: 4096
  max_response_length: 1024

trainer:
  n_gpus_per_node: 1
```

```yaml
  total_epochs: 3
  save_freq: 50
  project_name: agentflow-qwen35-lora
  experiment_name: qwen35-0.8b-flow-grpo
```

```bash
sed -i 's|train/config.yaml|train/config_qwen35_lora.yaml|g' train/train_with_logs.s
```

5.3 启动训练（两个 tmux 窗口）

```bash
# 窗口 0: AgentFlow service (只调 API, 不占 GPU)
tmux new-session -s agentflow
source .venv/bin/activate
bash train/serve_with_logs.sh

# 窗口 1: verl 训练
tmux new-window
source .venv/bin/activate
wandb login
export WANDB_PROJECT=agentflow-qwen35-lora
bash train/train_with_logs.sh
```

5.4 合并 LoRA 权重

```python
# scripts/merge_lora.py
from peft import PeftModel
from transformers import AutoModelForCausalLM, AutoTokenizer

base_id = "Qwen/Qwen3.5-0.8B"
lora_ckpt = "./checkpoints/qwen35-0.8b-flow-grpo-final"
output_dir = "./checkpoints/qwen35-0.8b-merged"

base = AutoModelForCausalLM.from_pretrained(base_id, torch_dtype="bfloat16")
model = PeftModel.from_pretrained(base, lora_ckpt)
merged = model.merge_and_unload()
merged.save_pretrained(output_dir)
AutoTokenizer.from_pretrained(base_id).save_pretrained(output_dir)
print("Done:", output_dir)
```

```bash
python scripts/merge_lora.py
huggingface-cli upload <your-username>/qwen35-0.8b-agentflow ./checkpoints/qwen35-0.
```

5.5 部署并重跑论文 5 个 benchmark

```bash
export MODAL_PLANNER_URL="https://...qwen35-0.8b-flow-grpo...modal.run"
cd test
bash bamboogle/run.sh && bash 2wiki/run.sh && bash hotpotqa/run.sh \
  && bash musique/run.sh && bash gaia/run.sh
```

Step 6：Flow-GRPO LoRA 训练后跑 Spider

> 对应作业要求第 6 条：用训练后的 Qwen3.5-0.8B 跑新 Benchmark，与 Step 4 无训练结果对比

```bash
# MODAL_PLANNER_URL 已指向训练后模型（同 Step 5.5）
export MODAL_PLANNER_URL="https://...qwen35-0.8b-flow-grpo...modal.run"
python test/text2sql/agentflow_spider.py
```

结果记录到 Spider 结果表的 `Qwen3.5-0.8B + Flow-GRPO LoRA` 行，观察 Flow-GRPO 在 Text-to-SQL 上的迁移效果。

完整结果表

论文 5 Benchmarks

模型	Bamboogle	2Wiki	HotpotQA	Musique	Avg	GAIA
论文目标（Qwen2.5-7B，无训练）	58.4	60.0	51.3	19.2	47.2	17.2
Step 2 复现（Qwen2.5-7B，Modal vLLM）						
Qwen3.5-0.8B（无训练）						

模型	Bamboogle	2Wiki	HotpotQA	Musique	Avg	GAIA
Qwen3.5-2B（无训练）						
Qwen3.5-4B（无训练）						
Qwen3.5-9B（无训练）						
Qwen3.5-27B（无训练）						
Qwen3.5-0.8B + Flow-GRPO LoRA						

Spider Benchmark（Text-to-SQL，新任务）

模型	Execution Accuracy
Qwen3.5-0.8B（无训练）	
Qwen3.5-2B（无训练）	
Qwen3.5-4B（无训练）	
Qwen3.5-9B（无训练）	
Qwen3.5-27B（无训练）	
Qwen3.5-0.8B + Flow-GRPO LoRA	

交付物

GitHub Repo README 结构

```
## Setup
- 环境安装（含 vLLM 主分支安装命令）
- API Keys 配置说明

## Step 1: Environment Setup
## Step 2: Reproduce AgentFlow (Qwen2.5-7B, no training)
## Step 3: Qwen3.5 Models (no training, 5 benchmarks)
## Step 4: New Benchmark — Text-to-SQL (Spider)
```

```
## Step 5: Flow-GRPO + LoRA Training (Qwen3.5-0.8B)
## Step 6: Trained Model on Spider
```

Presentation 大纲（5 分钟硬限制）

时间	内容
0:00–0:30	AgentFlow 架构一句话介绍
0:30–1:30	Step 2 复现结果 vs 论文目标
1:30–2:30	Step 3 Qwen3.5 各尺寸无训练结果
2:30–3:30	Step 4 Spider 新任务各尺寸结果
3:30–4:30	Step 5+6 训练前后对比
4:30–5:00	结论

最少 4 张幻灯片：架构图 / 论文 benchmark 汇总表 / Spider 结果表 / 结论。

常见坑

问题	原因	解决
Planner 输出不是合法 JSON	Qwen3.5 thinking mode 未关	所有 Qwen3.5 推理都加 `enable_thinking: false`
vLLM 加载 Qwen3.5 报错	用了标准 pip 版	改用主分支：`uv pip install vllm --torch-backend=auto`
Modal 第一次调用超时	cold start ~30s	`keep_warm=1` 保持实例热启
Executor/Verifier 报 rate limit	API 限速	加 retry，或 DashScope/Together 互为备用
verl LoRA 配置不生效	键名路径错误	确认在 `actor_rollout_ref.model` 下，不是顶层 `model`
serve 和 train 冲突	serve 意外占 GPU	确认 `serve_with_logs.sh` 只调 API，不做 GPU 推理

问题	原因	解决
Spider 数据库找不到	spider.zip 需手动下载	去 https://yale-nlp.github.io/spider/ 下载
solver.solve() 结果取法	真实 API 返回 dict	用 `output["direct_output"]` 取文本